MeetLoop 方向评估与深化安排

当前执行补充 v2.3 | RN 与 Swift 为核心 | 每周总投入更新为 21 小时起

结论：v2.2 的技术方向已经覆盖你的目标。保留产品与架构，提升验收深度，并为独立 Agent 项目重新分配时间。

已核对原计划的第 1、3-9、14-16 页。它已安排 SwiftUI 起步、UIKit 专项、RN 报名模块、Kotlin 原生宿主，以及 Go 单体与数据库。这里评估的是计划覆盖度；本次没有审查应用仓库，不能据此认定这些能力已经掌握。

方向	现有覆盖	本次强化
Swift / iOS	语言、状态、并发、UIKit、网络与性能	把独立交付、故障实验、测试与性能测量列为必过关卡。
TypeScript / RN	React 基础、报名模块、双端宿主集成	加深 JS 运行机制、原生模块契约、构建发布与跨层排错。
Kotlin / Android	Compose 小链路、协程、生命周期	掌握宿主重建与取消，不要求复制整个 iOS 产品。
Go / 后端	鉴权、SQL、事务、幂等	继续控制规模；弱化服务数量，不削弱数据正确性。

两份项目的关系

MeetLoop 保持“活动列表、详情、报名、我的报名”的客户端产品。AgentDesk 使用自己的工单与模拟业务系统，仓库、数据库、部署与验收独立；只共享 Git、HTTP、SQL、测试和 React 等知识。

原 16 页计划继续提供课程和功能细则。本补充优先决定时间分配、阶段关卡与求职节奏。原文件和历史版本均保留。

iOS 主项要学到哪一层

能独立实现、解释、诊断和验证；“看完课程”只算输入

能力	必须能解释	用 MeetLoop 验收
Swift 基础	值与引用、Optional、协议、泛型、闭包捕获、ARC	不用教程增加报名规则；修复一次循环引用，解释 weak 与所有权。
SwiftUI 状态	状态来源、身份、依赖、视图更新、导航	筛选与页面往返不丢状态；解释谁持有数据，定位多余更新。
UIKit 专项	控制器生命周期、约束、列表复用、事件传播	复用同一数据层实现列表 / 详情；修复图片错位和退出后仍回调。
并发与网络	async / await、任务取消、actor 隔离、错误传播	快速切页与旧请求晚到；主线程检查；超时后查询报名真实结果。
数据与可靠性	缓存有效期、持久化迁移、令牌安全、重试边界	离线与登录失效能解释和恢复；服务端结果是最终依据。
调试与性能	断点、日志、Memory Graph、Instruments	固定设备、构建和数据量，记录一个瓶颈的定位与改前改后结果。
测试与交付	依赖替换、单元 / 集成 / 核心 UI 测试、签名	核心流程 Release 可运行；故障回归进入 CI，写清构建与运行步骤。

每个专题都留四样证据

- 一个可复现的问题；一段自己的原因解释；一次最小修复；一项能防回归的验证。
- 允许用 AI 查资料、补脚手架和审查，但状态设计、取消规则、所有权和关键测试要自己先写方案。
- Objective-C 互操作按目标 JD 补一个小实验；不要为追技术词条提前扩展大型架构。

Swift 语言指南 / 100 Days of SwiftUI

Apple Today / UIKit / Stanford CS193p

RN 主项与 Android 辅项的深度

先懂 JS / TS / React，再学 RN；跨端不是只会写 JSX

层次	必做实验与交付
JS / TS / React	解释事件循环、Promise、闭包与旧状态；用联合类型表示报名结果；修复一次 effect 清理问题，独立改表单规则。
RN 页面与状态	表单、列表、键盘、返回、请求取消、可访问性；区分 JS 计算、原生布局与主线程阻塞，测量实际问题。
宿主集成	输入活动 ID 与会话能力，输出完成 / 取消 / 需登录 / 待确认；原生负责导航，规定回调次数、生命周期和刷新。
原生能力	完成一个系统分享适配；按所用 RN 版本理解 Codegen、TurboModules、Fabric、JSI 的职责，不把旧 Bridge 教程当默认。
Kotlin / Android	空安全、协程 / Flow、ViewModel、Compose 状态、Activity 重建；处理旋转、返回键、重复打开与宿主销毁。
构建与发布	锁依赖和工具链；iOS / Android Release 不依赖 Metro；验证权限拒绝、构建失败、错误定位与版本回退。

Go 保留六件事

HTTP 契约与错误码；认证和资源权限；输入校验；数据库迁移与约束；报名事务与幂等；日志、备份和最小部署。用并发请求验证不超卖、不重复占名额。暂不加入 Kafka、微服务与 Kubernetes。

RN → 原生 → Go API → 数据库 → 回调刷新：能沿一条链路排错，才算跨端主项过关。Android 学习范围可以小，边界行为必须完整。

React Learn / TypeScript Handbook

React Native 现有应用集成 / React Native 原生模块

Android Basics with Compose / A Tour of Go

按每周 21 小时同时推进

2026 年 10 月 28 日更新 | 课程、调试、文档和面试训练均计入预算

时段	MeetLoop	AgentDesk	公共基础 / 复盘
起步与春季实习准备	11 小时	7 小时	3 小时
iOS 小链路稳定后	10 小时	8 小时	3 小时
某一项目专项攻坚	8 / 12 小时	10 / 6 小时	3 小时

起步可安排周一至周五各 3 小时，周末合计 6 小时。MeetLoop 的 11 小时约为课程 3 小时、实现 6 小时、调试验证 2 小时；Agent 的 7 小时约为基础 / 源码 2 小时、实验 4 小时、评测 1 小时。额外时间先用来复现与独立改题。

日历参考	客户端主线与阶段结果
2026.11-12	Swift / SwiftUI；本地四步小链路；12 月开始让 2-3 人体验并记录问题。
2027.01-02	网络、取消、持久化、UIKit 专项；最小 API；准备一个可演示的 iOS 实习作品。
2027.03-06	若入职，实习与学校占用优先；个人项目按真实空闲调整。iOS 过关后开始 JS / TS / React，再进入 RN。
2027.07-10	RN 集成 iOS，再接 Android 宿主；完成跨层调试、Release 和一个原生能力。
2027.11-2028.04	性能、可靠性、两轮用户反馈与求职；按目标地区选择 iOS 或 Agent 作为面试主项目。

月份是参考，不等于自动过关。此前每周 12-15 小时的限制已被你最新的 21 小时起覆盖；若实习、搬家与考试改变实际可用时间，就延长日历，不删核心验收。

两个项目都做深，但不要同一周同时打开 SwiftUI、RN、Kotlin、Python 四套入门主课。先稳定一个阶段，再增加下一层。

前四周与春季实习最小成果

从当前掌握程度开始；已经独立通过的内容可跳过，不必重新刷课

周次	MeetLoop 11 小时如何用	验收
第 1 周	确认 Xcode / Git；100 Days 当前基础章节；变量、集合、函数小练习；建立 3 条活动数据。	关闭教程，改活动字段与筛选条件；能解释代码。
第 2 周	SwiftUI List、详情与导航；状态来源；实现内存报名 / 取消。	往返与重复点击不重复占名额；自己画数据流。
第 3 周	我的报名与四态；Repository Mock；加入延迟、空数据和错误开关。	独立增加一条报名规则；用断点定位一次错误。
第 4 周	按基础补并发 / 取消；测试关键状态；第一次用户体验；录制演示。	修复用户卡点；保存至少一项故障复现与回归证据。

若尚不理解结构体、闭包或状态，上表顺延：先完成对应课和小练习。第一天 90 分钟足够做环境与变量任务；不要同时开始 Today 和 Stanford 两套长课。

2027 年 2 月底争取具备

- 一个可运行的 iOS 报名小链路，能当场改规则并解释错误、取消与数据状态。
- 一处 UIKit 专项、一次内存或并发故障定位、关键业务测试和一段清晰演示。
- 能说明字节实习中自己真正理解和完成的部分；AI 参与的工作不包装成未掌握的能力。

上海、每周 4-5 天、连续至少 3 个月，是目前的求职约束。暂不以回字节为目标；学校身份与实习时长按具体 JD 核对。RN 尚未完成就如实写学习中，不为了春季投递仓促拼三端。

100 Days of SwiftUI / Apple Today / UIKit

怎样验收为真正掌握

统一采用解释、实现、变更、故障、测量五种证据

关卡	现场任务	通过标准
G1 原生基础	40 分钟增加活动筛选并保留状态	无整段 AI 代写，能说明每处状态与边界。
G2 可靠性	制造旧响应晚到、超时与重复请求	从日志 / 断点找原因；修复后关键用例可重复。
G3 跨端集成	RN 成功后原生页不刷新	解释输入输出契约，修复且不引入重复回调。
G4 双端宿主	Android 重建或 iOS 页面退出	状态与任务生命周期合理；Release 可运行。
G5 工程深化	选择一个真实性能或内存问题	有固定条件、原始数据、取舍与回归检查。

使用 AI 的训练方式

先自己写需求理解、状态图和验证清单，再让 AI 评审或提示；卡住时先拿日志与最小复现求助。完成后关掉对话，换一个条件重新实现。每周至少保留一次不依赖整段生成的练习，并记录自己尚不能解释的代码。

最终作品包

- 运行说明、架构与接口契约、3 分钟演示、核心测试、两轮反馈、3 个工程案例。
- 案例写“触发条件 → 观察 → 假设 → 验证 → 修复 → 代价”；没有测量的提升不写百分比。
- 不以读完框架源码为终点：能沿业务调用链追到关键实现，并通过小实验验证判断。

这份计划覆盖移动端与跨端岗位的主要训练方向，但不等于完整 Web 前端训练。若目标转为纯 Web 岗，需要另补浏览器、DOM / CSS、Web 性能与安全。

课程使用与资料优先级

保留一门当前主课；其他资料解决眼前的问题

阶段	使用资料	停止看课、返回项目的时机
SwiftUI 起步	100 Days of SwiftUI + Swift 语言指南	完成一个检查点就把同一概念迁移到活动业务。
UIKit 专项	Apple Today	完成列表、导航和复用实验后回到原项目。
SwiftUI 深化	CS193p 选学并完成对应作业	能解释模型、状态和身份；不与主课全量并行。
RN 准备	TypeScript Handbook + React Learn	独立实现表单、异步请求、错误与清理。
跨端集成	RN 官方集成与原生模块文档	形成宿主契约、双端 Release 与故障记录。
Android 辅修	Android Basics with Compose	够用的 Kotlin、生命周期、协程和宿主链路。
最小后端	A Tour of Go + 实际接口 / SQL	能读懂服务端权限、事务和幂等测试。

工具链以设备支持的正式版为准，并记录 Xcode、Swift、RN、Node、Android Gradle Plugin 与 JDK 版本。不强行沿用旧教程的工程配置。React 概念可以共享，React DOM 与 RN 组件 / 样式体系仍需分别理解。

100 Days of SwiftUI / Swift 语言指南

Apple Today / UIKit / Stanford CS193p

React Learn / TypeScript Handbook

React Native 现有应用集成 / React Native 原生模块

Android Basics with Compose / A Tour of Go

资料状态：原课程入口从 v2.2 保留；本次重新核对了 RN 集成与原生模块文档。历史 PDF 中的招聘、硬件和版本信息不是本次重新验证的结论。